Applying Security Best Practices in Docker

In this train, we'll learn how security best practices can be applied for containers, including secure image creation, container isolation, and least privilege principles. You'll also learn how to reduce attack surfaces, enforce isolation, and apply runtime restrictions that harden your deployments.

Learning objectives

> - Apply security best practices for containers, including secure image creation, container isolation, and least privilege principles.
> - Limit container permissions, isolate services, and configure safer runtime environments.
> - Use tools like Docker CLI and Compose to apply these techniques in your own projects.

Secure Image Creation

When creating Docker images, security should be a priority. Start by using trusted and minimal base images. For example, prefer using official images or lightweight distributions like Alpine over heavier images like Ubuntu or Debian. This reduces the number of vulnerabilities that could be present in your images.

```
# Risky: Too many unnecessary packages
FROM ubuntu:latest

# Recommended: Lightweight and secure
FROM python:3.11-slim
```

Explanation:
`FROM` specifies your base image. Avoid using `:latest` which may pull unstable versions. Instead, pin to a versioned, slim image for reproducibility and smaller attack surface.

Only install the packages your application needs. Always clean up to reduce the image size and avoid leaving behind potential exploits.

```
RUN apt-get update && \
    apt-get install -y curl git && \
    apt-get clean && rm -rf /var/lib/apt/lists/*
```

Explanation: The `apt-get clean` and `rm -rf` help reduce image size and remove sensitive install traces.

Use Non-Root Users

```
RUN addgroup --system appgroup && \
    adduser --system --ingroup appgroup appuser

USER appuser
```

Why, running as root inside containers is dangerous — if compromised, an attacker can gain control of the host system. Always use a non-root user.

Scan for Vulnerabilities

Use tools like `docker scan` or `trivy`:

```
docker scan myapp
trivy image myapp
```

These tools help identify known vulnerabilities (CVEs) inside your Docker images before deployment.

Container Isolation

To prevent one compromised container from affecting others, ensure containers are properly isolated.

- **Resource Limits:** Use Docker flags like `--memory` and `--cpus` to prevent resource exhaustion.
- **Namespaces:** Docker leverages Linux namespaces (PID, net, mount, etc.) to isolate containers from each other and the host.

Use Custom Networks

Limit container communication using user-defined bridge networks:

```
# Create a private network
docker network create secure-net
```

Example from Docker Compose:

```
# docker-compose.yml
services:
  app:
    image: myapp:secure
    networks:
```

```
      - secure-net
  db:
    image: postgres:15-alpine
    networks:
      - secure-net

networks:
  secure-net:
    driver: bridge
```

Containers outside this custom network cannot communicate with these services, enforcing isolation.

Read-only File Systems

```
docker run --read-only -v /app/data myapp
```

Why? This prevents attackers from writing or modifying files in the container. Use volumes for write access only when necessary.

Least Privilege Principle

Grant containers only the permissions they need to function — no more.

- **Avoid running as root:** Create non-root users with `USER` directive in your Dockerfile.
- **Limit capabilities:** Drop all unnecessary Linux capabilities.
- **Read-only file systems:** Mount root as read-only to prevent tampering.

Example: Drop Capabilities

```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE myapp
```

Explanation: This drops all default Linux capabilities and re-adds only what's needed (like binding to port 80).

Use Security Profiles

```
docker run --security-opt no-new-privileges \
           --security-opt seccomp=default.json \
           --security-opt apparmor=docker-default myapp
```

Explanation:

- `no-new-privileges`: Prevents the container from gaining more permissions during runtime.
- `seccomp`: Restricts system calls the container can make.
- `apparmor`: Applies mandatory access controls.

Conclusion

By following these security best practices, you ensure that your Docker containers in data engineering workflows are secure, efficient, and production ready. Security isn't a one-time task it’s an ongoing process of hardening, reviewing, and monitoring.

Key Takeaways ✖

- **Use minimal base images:** Keep images lightweight and reduce vulnerabilities.
- **Use resource limits and namespaces:** Prevent containers from interfering with each other.
- **Avoid running containers as root:** Always create non-privileged users.
- **Limit container capabilities:** Drop unnecessary privileges using `--cap-drop`.
- **Apply runtime restrictions:** Harden containers with seccomp, AppArmor, and read-only file systems.